Sparse View Plant

Project Profile Document

Sparse-view 3D reconstruction of plants — uncalibrated RGB photos to a point cloud and refined 2D Gaussian geometry

Version 0.1.0 · 2026-08-06 · MIT License

---

1. Project Background

Reconstructing plants in 3D is notoriously difficult for classical structure-from-motion (SfM) and multi-view stereo (MVS) pipelines: thin stems and leaves, repetitive and specular surfaces, textureless regions, and heavy self-occlusion break keypoint matching. Conventional capture also requires a calibrated camera rig or an offline bundle-adjustment stage such as COLMAP, and often per-view foreground masks.

This project targets the sparse-view, uncalibrated, mask-free setting: given only **3–6 ordinary RGB photos** of a single plant, it produces a fused 3D point cloud and a refined surface-aligned 2D Gaussian model. No camera intrinsics, extrinsics, calibration metadata, or masks are required at inference time.

The system combines a feed-forward visual-geometry foundation model (VGGT) for geometry and camera estimation with a lightweight per-scene photometric refinement of 2D Gaussian surfels (`gs-refine`, built on `gsplat`). No training is required at deployment; all model weights are downloaded from Hugging Face Hub on first use. Design goals are cross-species generalization, geometry-first quality (topology and thin structures over texture), and lightweight inference suitable for eventual edge deployment.

2. Target Users & Application Scenarios

The primary users are researchers and practitioners who need fast, non-destructive 3D plant models from casual photography:

- **Plant phenotyping researchers** — quick 3D capture from handheld or fixed-phone photos for trait measurement, growth analysis, and architecture studies.
- **Agronomists and breeders** — non-destructive monitoring of plants over time in greenhouses or field trials, without specialized capture rigs.
- **Horticulture and botanical collections** — digitizing individual specimens for documentation and virtual archives.
- **3D content and digital-twin builders** — generating plant geometry for virtual scenes from a handful of photos.

Typical usage modes are an interactive browser demo (Gradio app) for single plants and a command-line pipeline for batch runs and evaluation. Inputs are one plant per run, roughly centered in the frame, captured from spread viewpoints; no calibration or pose metadata is available to the deployed inference path. The project is research-stage software: reported results are input-view rendering metrics, not a production or clinical claim.

3. System Architecture

The software is a single pipeline exposed through two interfaces: a Gradio web app (`scripts/gradio_app.py`, Live Reconstruction workflow) and a CLI (`scripts/run_pipeline.py`). Configuration is TOML-based (`configs/`), the runtime is Python 3.12 managed with `uv`, and per-run artifacts are stored under `outputs/`.

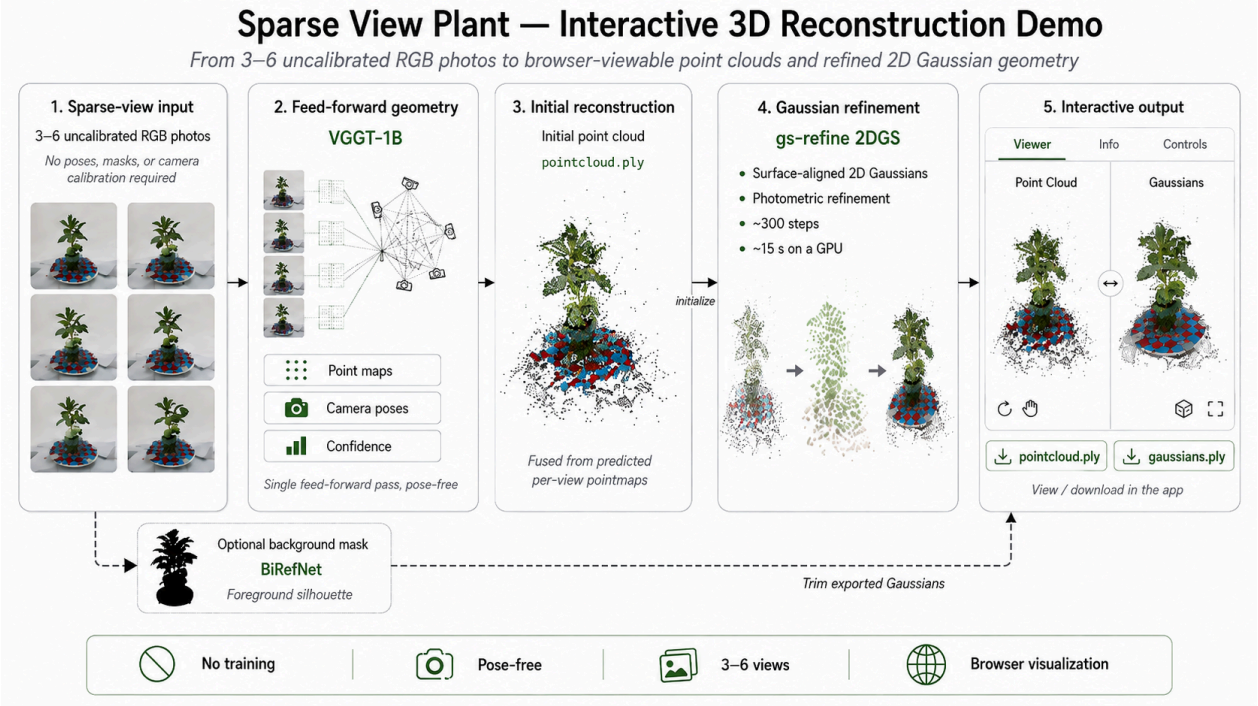


Figure 1: End-to-end pipeline: uncalibrated photos to fused point cloud and refined 2D Gaussians.

The stages are:

1. **Input** — 3–6 uncalibrated RGB images of one plant (folder upload in the app, or `--images <folder>` in the CLI).
2. **Point-cloud stage (VGGT frontend)** — a feed-forward pass predicts per-view world pointmaps, camera poses, and point confidence; per-view intrinsics are recovered from pixel/3D pointmap correspondences rather than assumed. A confidence percentile filter keeps the top 80% of points, and an optional probe pass selects the most spread views (FPS/PCA wide-baseline selection).
3. **Foreground stage (optional)** — per-view background masks weight later supervision and trim exports. The default provider is BiRefNet salient segmentation; a mask-free fallback (central ellipse + per-view confidence quantile) requires no extra weights.
4. **Gaussian refinement stage (`gs-refine`)** — the fused cloud initializes surface-aligned 2D Gaussian surfels, refined for 300 steps under a mask-weighted photometric loss against the input views, rendered with `gsplat.rasterization_2dgs`.
5. **Outputs** — `pointcloud.ply`, `gaussians.ply`, `pipeline.json`, and a run log, viewable and downloadable from the Gradio Model3D viewers.

4. Model & Algorithm Introduction

VGGT (frontend). VGGT (CVPR 2025, arXiv:2503.11651) is a feed-forward visual-geometry transformer: a DINOv2 backbone with alternating self- and cross-frame attention predicts per-view camera poses, intrinsics, depth, and dense 3D world pointmaps from an unordered image set in a single pass. This project uses the 1B checkpoint (`facebook/VGGT-1B`, 518 px training resolution; 448 px on 16 GB GPUs). Point confidence is mapped monotonically to [0, 1] and used for quantile-based foreground and opacity filtering.

Pointmap intrinsics. Instead of assuming a focal length, each view's intrinsic matrix is recovered from the pixel/3D correspondences in the predicted pointmap grid, which stabilizes refinement on real captures.

Surface-aligned 2DGS initialization. For every pointmap cell, the two image-space tangent directions define the local surfel frame, and the cross-product normal is flipped toward its source camera. Surfels are initialized as thin disks (`refine_scale_disk = 1e-3`) so that leaf orientation is preserved from the geometry prior instead of being recovered from random rotations.

gs-refine (backend). A standard 3DGS-style model is optimized directly from the frontend cloud: means from points, scales from local density, opacity from confidence logits, colors from fused point RGB (SH DC), and — in the 2DGS variant — trainable quaternions for disk orientation. All attributes move under a mask-weighted photometric loss against the input views rendered with the frontend cameras: L1 color, alpha, background, and depth-distortion terms (`refine_bg_weight = 0.5`, `refine_alpha_weight = 0.1`, `refine_dist_weight = 0.1`). Density control uses `gsplat.strategy.ops` (`duplicate`/`remove`/`split`/`reset_opa`), capped at 400 k points; a run takes roughly 30–60 s on a Tesla T10.

BiRefNet (optional mask). BiRefNet (220 M parameters, fp16) provides salient-foreground masks from the cached Hub checkpoint (`ZhengPeng7/BiRefNet`). Empty-mask views fall back to a cross-view central-ellipse prior.

Evaluation. Metrics are input-view rendering PSNR, SSIM, and silhouette IoU at 256 px, under identical VGGT prediction, evaluation cameras, pointmap intrinsics, opacity threshold, and native 2DGS renderer, on the Bean 4 set:

Method	PSNR ↑	SSIM ↑	Silhouette IoU ↑
VGGT + vanilla 2DGS	15.70	0.169	0.389
Plant-aware 2DGS (ours)	17.45	0.215	0.406
Δ	+1.75	+0.046	+0.017

Table 1: Input-view rendering comparison on the Bean 4 set.

5. Adaptation for AMD Radeon GPU / ROCm

Upstream `gsplat` is CUDA-only, so an AMD Radeon GPU requires a small stack adaptation. Two components must be swapped for HIP-native builds; the application code and `pyproject.toml` remain unchanged.

ROCm PyTorch. PyTorch is installed from AMD's multi-arch wheel index with a `device-gfxXXXX` extra that selects the build matching the GPU architecture. For the AMD Radeon Pro W7900D (`gfx1100`, RDNA 3):

```
uv pip install --index-url https://repo.amd.com/rocm/whl-multi-arch/ \
    "torch[device-gfx1100]==2.12.0+rocm7.14.0" \
    "torchvision[device-gfx1100]==0.27.0+rocm7.14.0" \
    "torchaudio==2.11.0+rocm7.14.0"
```

AMD's HIP port of gsplat. AMD publishes `amd_gsplat` on its own PyPI (`pypi.amd.com`, ROCm 6.4.3 / 7.0.0 simple indexes). The wheel installs the `gsplat` Python package, so `import gsplat` and all call sites stay the same:

```
uv sync --extra gpu \
    --no-install-package torch \
    --no-install-package torchvision \
    --no-install-package gsplat
uv pip install --index-url https://pypi.amd.com/rocm-7.0.0/simple/ \
    --no-deps amd_gsplat
```

The `gpu` extra pins CUDA wheels (`torch>=2.13.0`, `torchvision>=0.28.0`), which is why the AMD wheels are installed manually and the matching packages are skipped during `uv sync`. Run the demo without re-syncing:

```
uv run --no-sync python scripts/gradio_app.py
```

API coverage. The API surface used by this project — `gsplat.rasterization`, `gsplat.rasterization_2dgs` (RGB+ED render modes with depth distortion), and `gsplat.strategy.ops` (`duplicate`, `remove`, `split`, `reset_opa`) — is implemented in the AMD port, which includes 2DGS tests. Upstream features not supported by the port (packed distributed training, PNG-compressed splat parameters, fused SSIM/Bilagrid) are not used here. Under ROCm PyTorch the `torch.cuda.*` APIs map to HIP, so the frontend (VGGT) and BiRefNet run unchanged.

VGGT optimization status. A MIGraphX graph-optimized build of VGGT was prepared for additional AMD acceleration, but it was not uploaded to Hugging Face due to time constraints. The deployed pipeline therefore uses the original PyTorch VGGT (unchanged) on both CUDA and ROCm.

Matching your GPU. Check the target architecture with `rocminfo | grep gfx` and adjust the `device-gfxXXXX` extra (`gfx1100` = Radeon Pro W7900D, `gfx942` = Instinct MI300X, and so on). AMD's official validation matrix covers Instinct datacenter GPUs; consumer Radeon cards are not part of the supported matrix. `gsplat` support on ROCm is best-effort and should be smoke-tested on the target machine before production use.